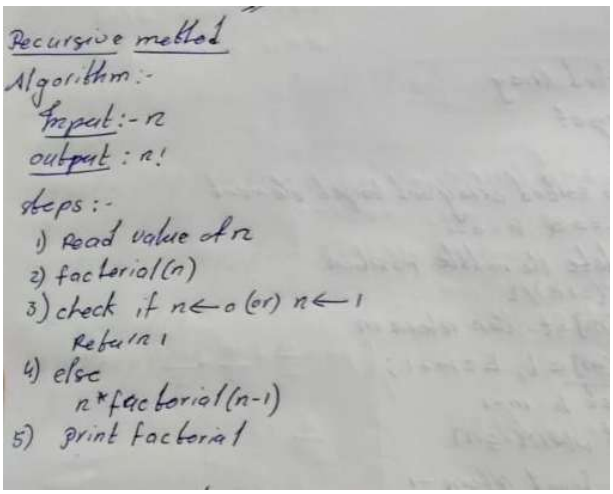
PRACTICAL: 04

AIM :- Implementation and Time analysis of factorial program using iterative and recursive method

Algorithms:- Recursive method

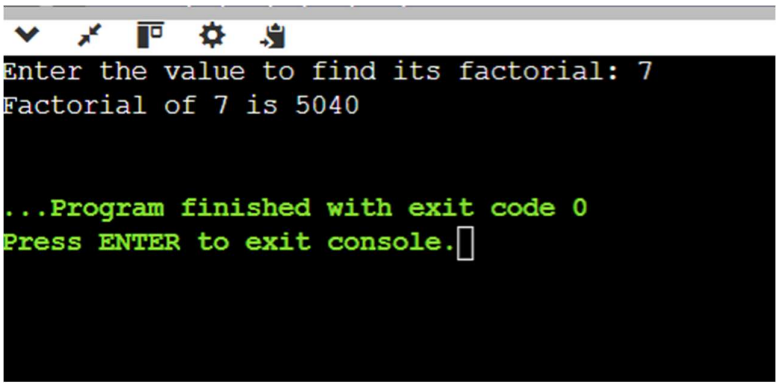


Program/code:-

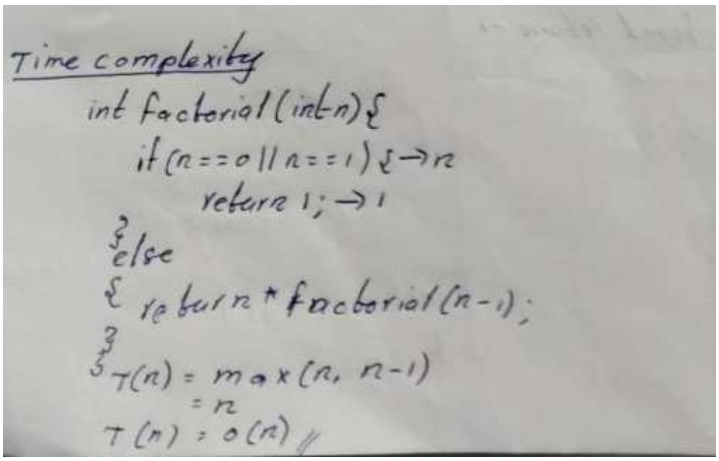
```
#include <iostream>
using namespace std;
class Factorial
{
public:
   int factr(int n)
   {
      if (n == 0 || n == 1)
      {
         return 1;
      }
```

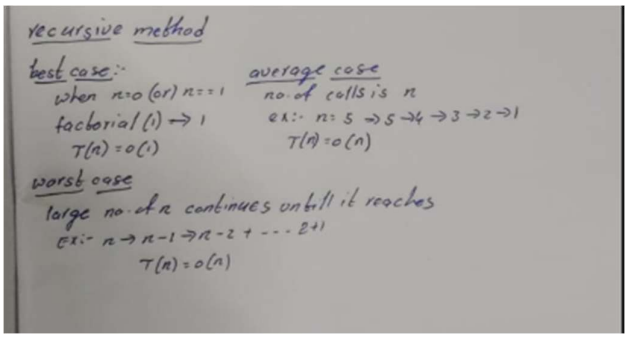
```
        else
        {
            int result = n * factr(n - 1);
            return result;
        }
    }
};
int main()
{
    int n, result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.factr(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:-

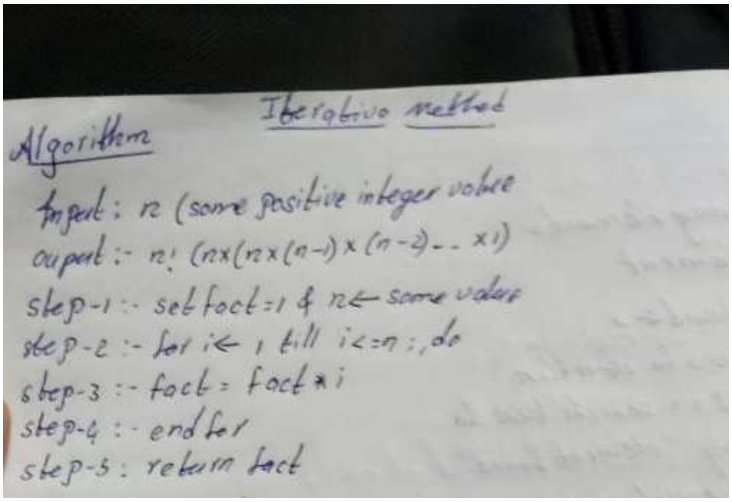


Timecomplexity:-





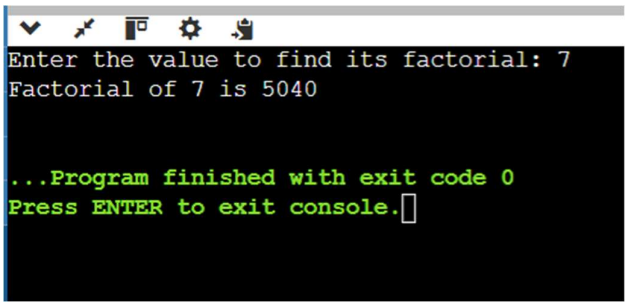
Algorithms:- Iterative method



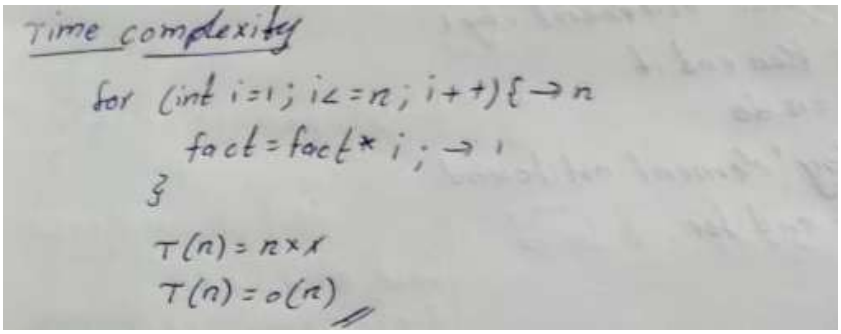
Program/code:-

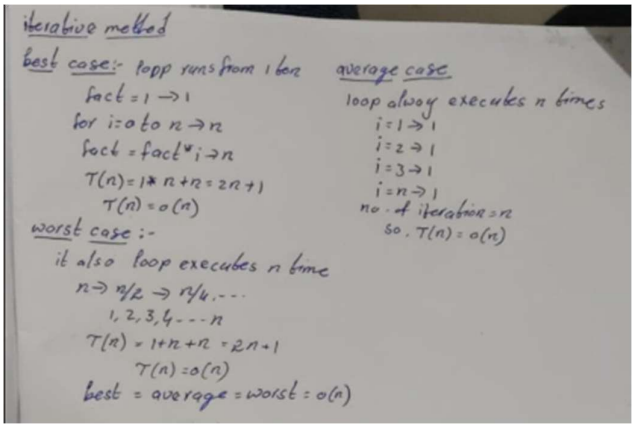
```
#include <iostream>
using namespace std;
class Factorial
{
public:
   int fact(int n)
   {
      int result = 1;
      for (int i = 1; i <= n; i++)
      {
         result = result * i;
      }
      return result;
   }
};
int main()
{
   int n,result;
   Factorial obj;
   cout << "Enter the value to find its factorial: ";
   cin >> n;
   result = obj.fact(n);
   cout << "Factorial of " << n << " is " << result << endl;
   return 0;
}
```

Output:-



Timecomplexity:-





Conclusion:-

The factorial program was implemented using both iterative and recursive methods. Both approaches produce the same result, but they differ in their execution and memory requirements.

The iterative method uses a loop to calculate the factorial and has a time complexity of O(n) and a space complexity of O(1). Therefore, it is generally more memory-efficient and suitable for larger values of n.